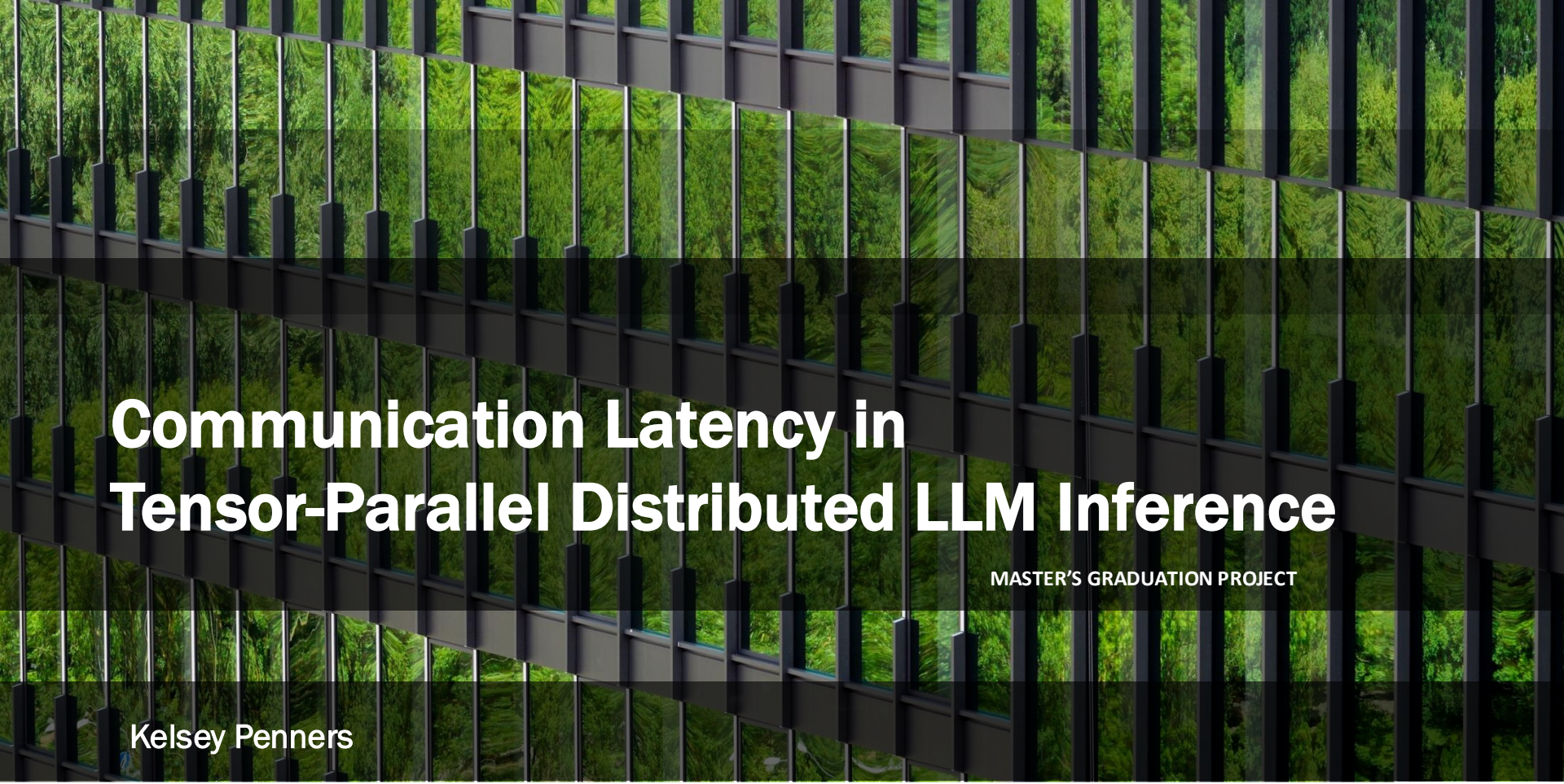


Communication Latency in Tensor-Parallel Distributed LLM Inference

MASTER'S GRADUATION PROJECT

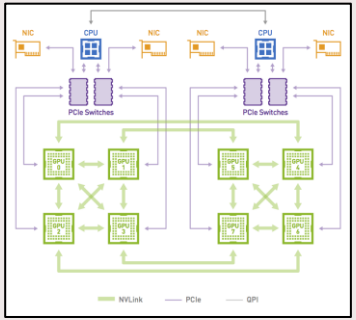
Kelsey Penners

Artificial Intelligence & Engineering Systems | Signal Processing Systems

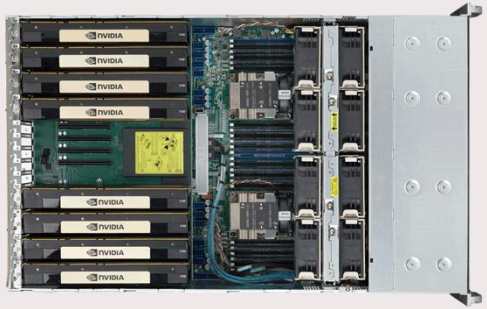
TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Distributing Large Language Models

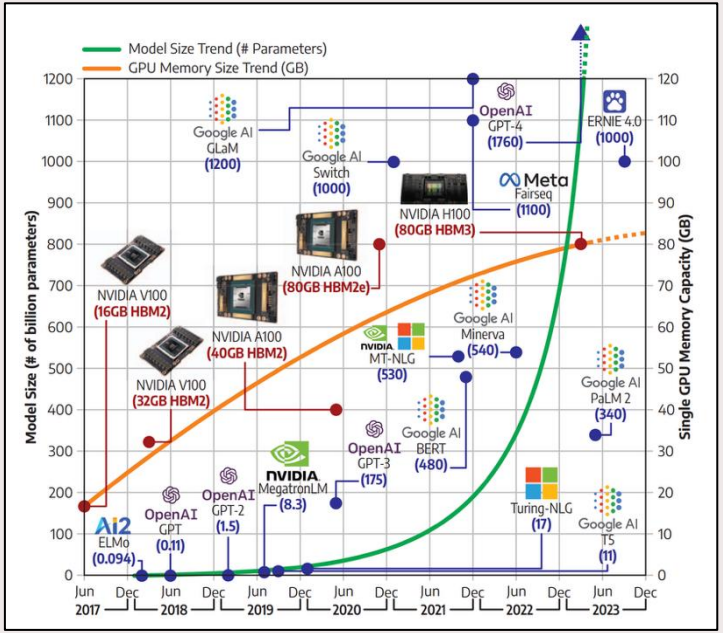
- LLM size is outpacing single-GPU memory capacity
  → Distribution is required
- Distribution introduces communication overhead from the synchronization of GPUs



[1]

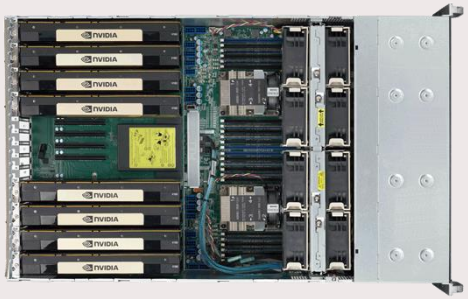


[2]



[3] Hanindhito et al.

State of the Art:
Communication in Distributed Training



[2]

- Communication occupies most of training time
  → collective operations can consume up to 90% of iteration time! [4] Qi et al.
- Network protocol matters
  → high-performance RDMA significantly outperforms others [4] Qi et al.
- Ethernet link latency has little measured impact on training performance [5] Mi et al.

Prior work is majorly training-focused.
Communication overhead in inference remains underexplored.

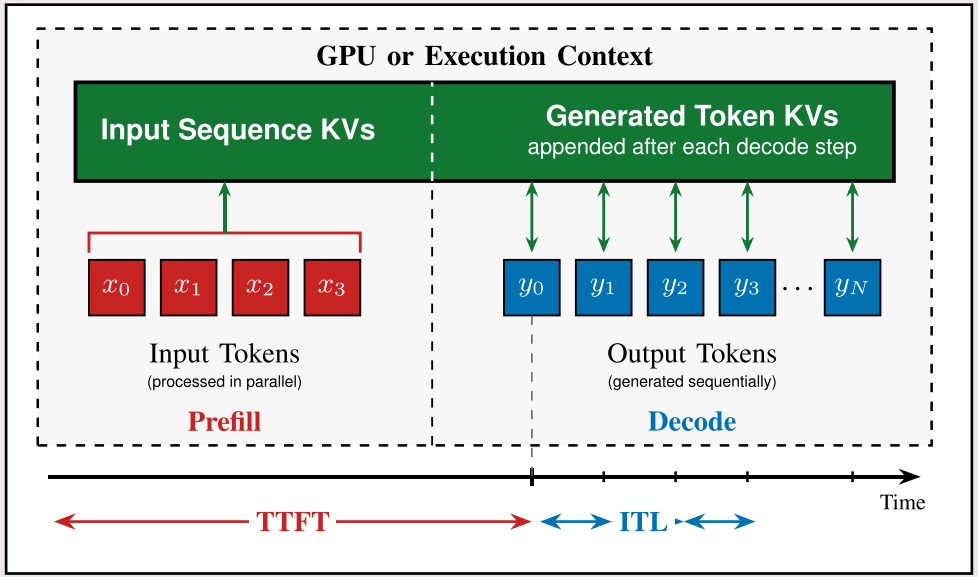
Research Questions

How does interconnect quality and communication latency affect tensor-parallel distributed LLM inference performance?

- How do the two inference phases differ in behavior under the same communication overhead?
- At what point does this overhead degrade performance enough to impact practical serving?

By investigating factors like performance latency, throughput, and energy.

LLM Inference Dynamics

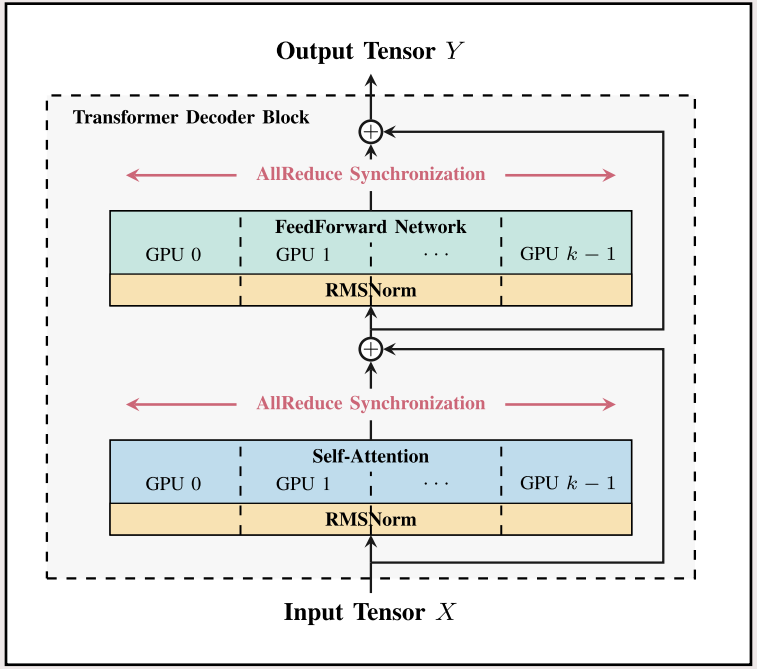


- **Prefill:** processes input sequence in parallel
- **Decode:** generates output tokens autoregressively, one step at a time.
- **Key-Value (KV) Cache:** stores all KV tensors

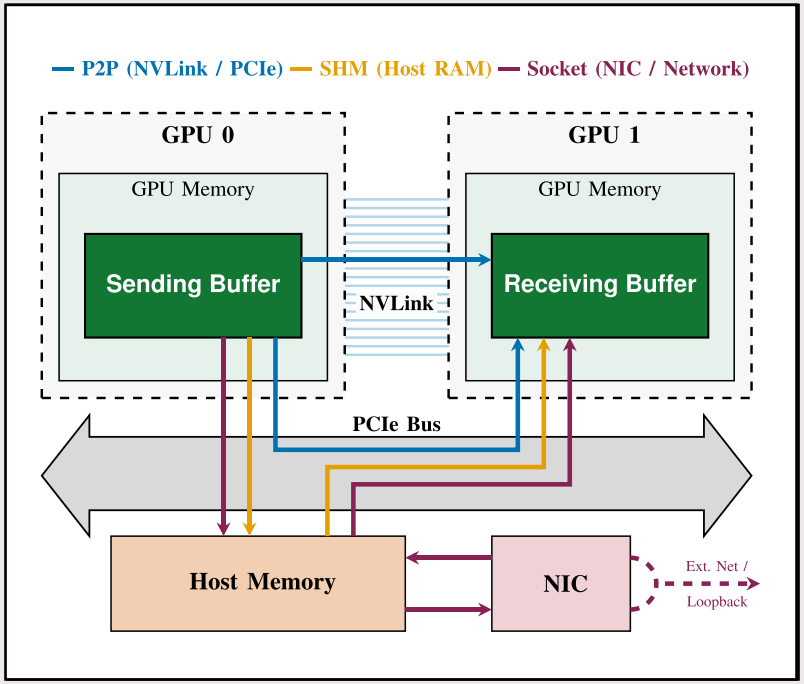
Key Metrics: Time to First Token (TTFT), Inter-Token Latency (ITL), Output Throughput (TPS), Energy per Token

Distributing using Tensor Parallelism

- **Tensor Parallelism (TP):** partitions each layer's weight matrices across *k* GPUs, typically within a node
- Requires two collective communication steps per transformer block: **AllReduce**
- AllReduce payload →
  [batch, sequence length, hidden dim]



GPU Communication: Transports & Interconnects



- GPUs interconnected by physical links: NVLink links and PCIe buses
- Communication traverses these links using **transports:**
  - Peer-to-Peer (P2P)
  - Host Shared Memory (SHM)
  - Socket

NVIDIA Collective Communication Library (NCCL)

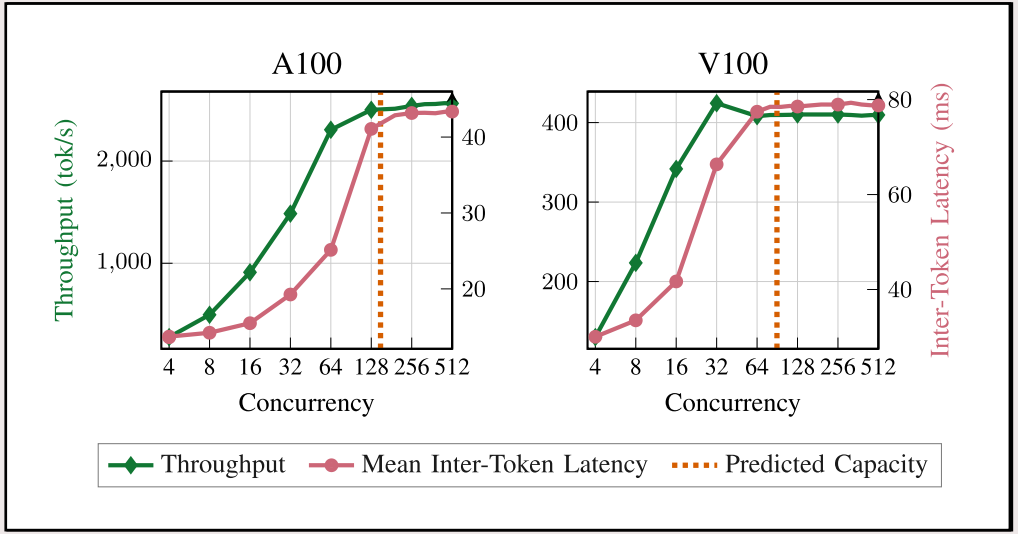
Methodology

- vLLM used as serving / benchmarking framework
- *Hardware*: A100-40GB and V100-32GB GPUs
- *Models*: Meta's open-source Llama models – parameter sizes {1B, 8B, 13B}
- Sweep across number concurrent requests
- Input/output sequence length = 512/512
- *Communication Transport Control*: environment variables
- *Latency injection:* into AllReduce calls

Spec.	A100 (Ampere)	V100 (Volta)
VRAM	40 GB	32 GB
Interconnect and Bandwidth	NVLink 3.0 600 GB/s	PCIe 3.0 32 GB/s
	PCIe 4.0 64 GB/s	
Mem. Bandwidth	1555 GB/s	900 GB/s
CUDA Compute Capability	8.0	7.0

Inference on a Single GPU

- vLLM engine preallocates KV cache based on VRAM
- Throughput and ITL plateau as the cache fills up, near predicted KV cache capacity



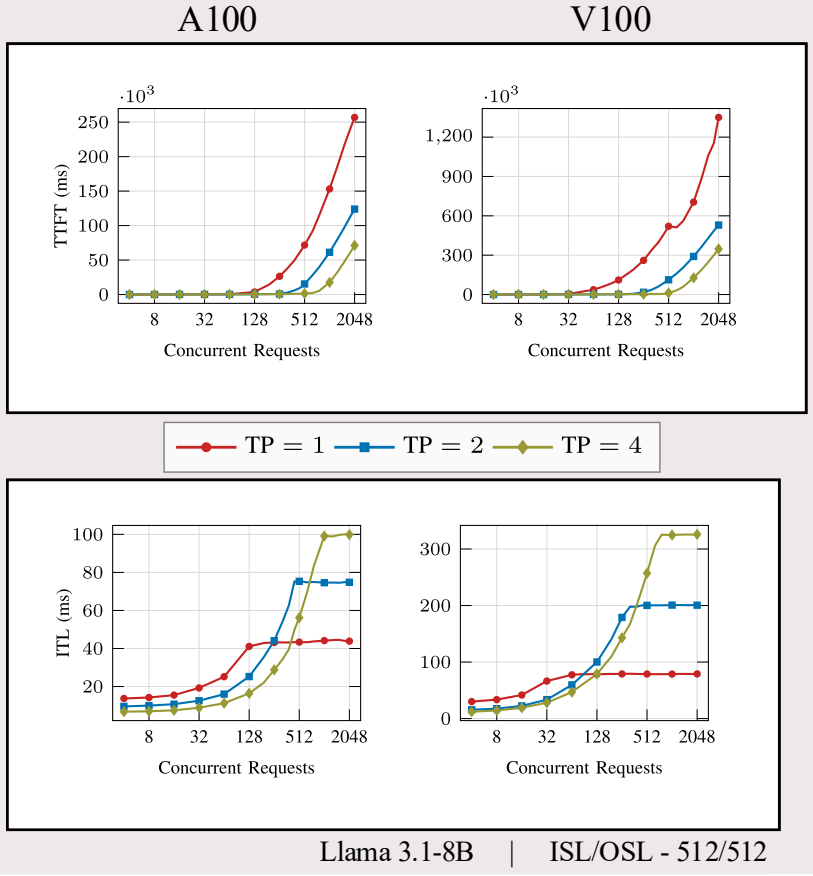
Llama 3.1-8B | ISL/OSL - 512/512

KV cache capacity sets the system limit: batching up to saturation trades higher performance latency for maximum throughput.

Scaling to Multiple GPUs

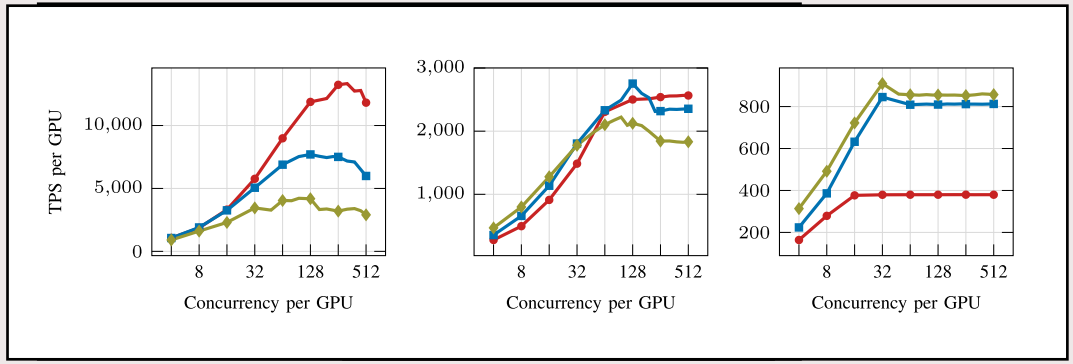
Latency Metrics

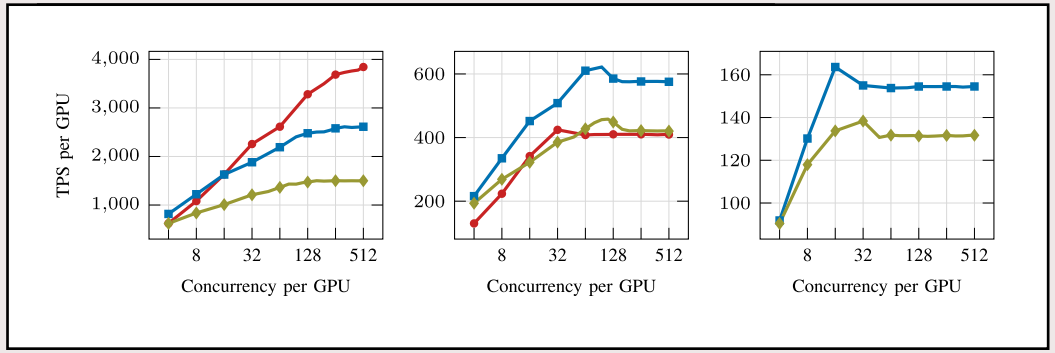
- KV cache capacity shifts as we scale to more GPUs
- Scaling speeds up forward passes under light workloads
- Older, PCIe-constrained V100 shows slower performance



TP scaling speeds up inference at light workloads and delays saturation.

Scaling to Multiple GPUs: Throughput Across Model Sizes





1B 8B 13B

- Model size sets memory pressure → determines scaling benefits
- Hardware determines additional compute / communication tradeoff

Optimal TP degree depends on memory pressure, hardware topology, and workload.

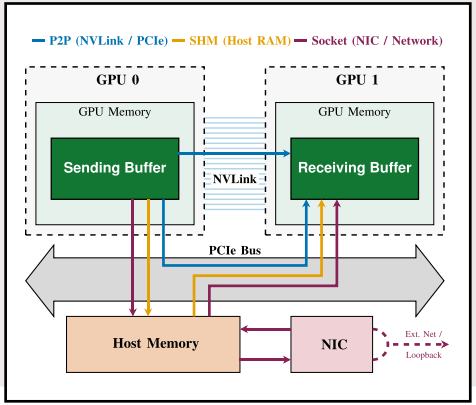


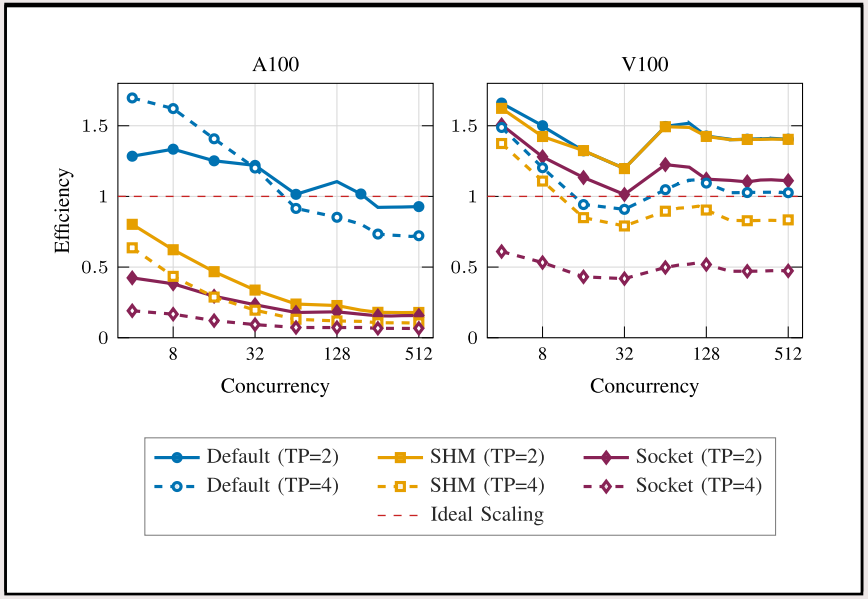
TU/e

Scaling with Different Interconnects & Transports

Throughput Scaling Efficiency

- *A100:* NVLink is the only path enabling efficient scaling
- *V100:* Default path uses PCIe, but seemingly scales better
- Memory-limited systems benefit more from distribution, even with slower transports





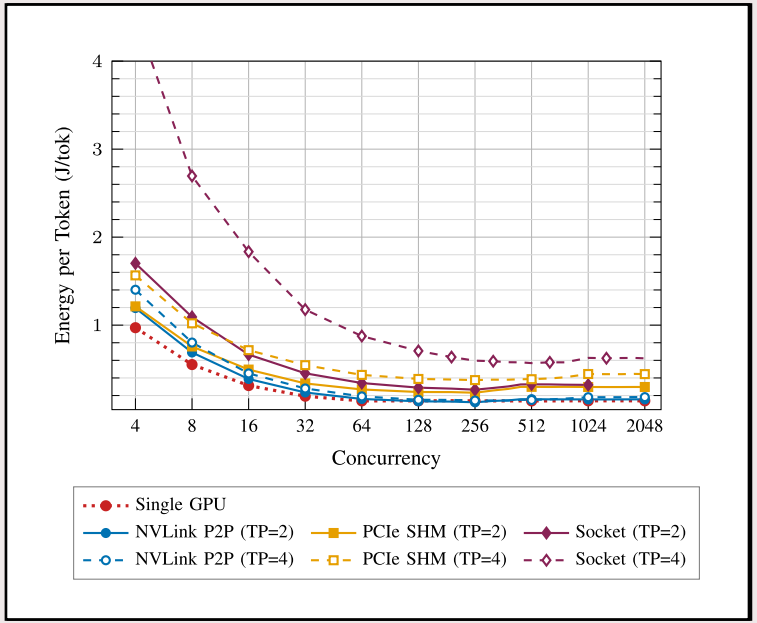
Llama 3.1-8B | ISL/OSL - 512/512

Sensitivity to transport quality is dependent on memory constraints.

Scaling with Different Interconnects & Transports

Energy Consumption

A100 System | Llama 3.1-8B | ISL/OSL - 512/512



- Energy per token decreases as GPU utilization saturates
- Lower-tier transports force slower synchronization of GPUs, driving up energy cost per token
- Scaling to more GPUs (higher TP) increases energy consumption.

Transport protocol directly impacts energy consumption.

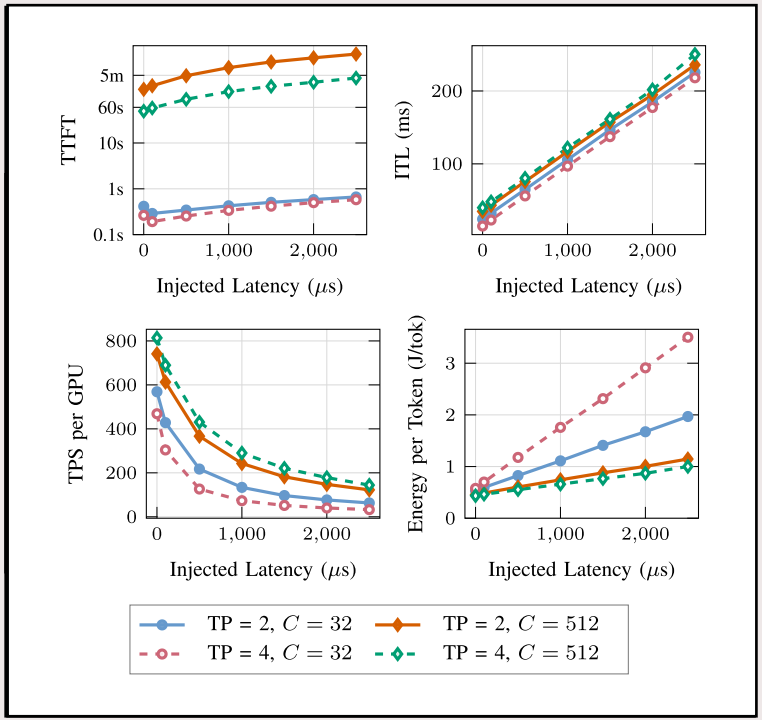
TU/e

Latency Injection:

How does the system respond to added communication latency?

- Injected directly into vLLM AllReduce calls via `torch.cuda._sleep()`
- ITL scales with model layers (L)
  → 2L × injected latency

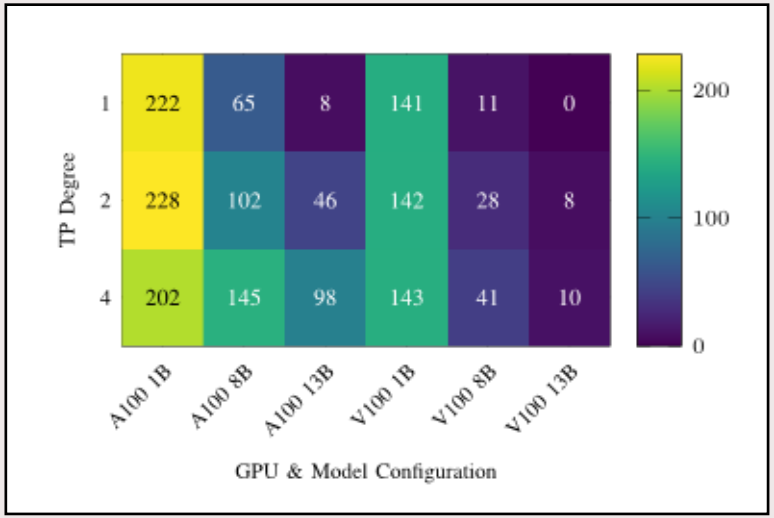
Communication latency compounds with model depth, system sensitivity depends on workload.



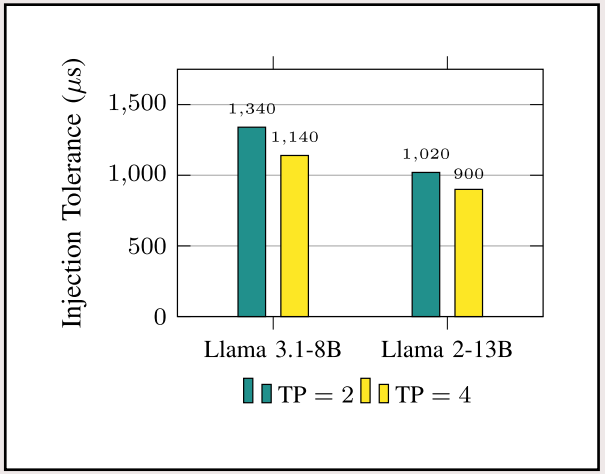
Practical Serving: Service Level Objectives (SLOs)

- Defined responsive SLOs at 90th percentile → (p90 TTFT ≤ 1s, p90 ITL ≤ 100ms)

SLO-Constrained Concurrent Capacity



Max Latency Injection under SLOs



Concurrency = 32

- Modern hardware enables higher concurrency while attaining targets
- Larger models constrain serving capacity and offer less latency headroom before violating SLOs

Key Takeaways & Insights

- **Hardware-Aware Tensor Parallelism:** Optimal TP configuration depends on physical interconnects, not just compute and memory budgets
- **KV Cache Capacity:** A saturated system maximizes throughput and minimizes energy per token at the cost of higher inference latency
- **Transport Fallback & AllReduce Latency Penalties:** Falling back from NVLink P2P to host-side transport paths or incurring collective communication latency slows execution, significantly lowering performance and resource efficiency

Future Work

- **Inter-Node Scaling:** evaluate inter-node inference with pipeline parallelism and across network fabrics (e.g., Ethernet)
- **Non-Dense Architectures:** quantify collective communication overhead in other models (e.g., Mixture of Experts or hybrid models)
- **New Hardware & Serving Optimizations:** extend to newer GPU architectures (e.g., NVIDIA Blackwell) and commonly used optimizations (e.g., quantization or prefill-decode disaggregation)

Thank You!

Questions?

Key Takeaways

- **Hardware-Aware Tensor Parallelism:** Optimal TP configuration depends on physical interconnects, not just compute and memory budgets
- **KV Cache Capacity:** A saturated system maximizes throughput and minimizes energy per token at the cost of higher inference latency
- **Transport Fallback & AllReduce Latency Penalties:** Falling back from NVLink P2P to host-side transport paths or incurring collective communication latency slows execution, significantly lowering performance and resource efficiency

TU/e

KV Cache Capacity Estimation

Formulas:

$$\text{Allocated KV cache} = \text{VRAM} \times \text{gpu_memory_utilization} - (\text{weights} + \text{activations} + \text{framework overhead})$$

$$\text{KV bytes per token} = 2 \times n_{\text{layers}} \times n_{\text{KV heads}} \times d_{\text{head}} \times d_{\text{dtype}} \quad [6]$$

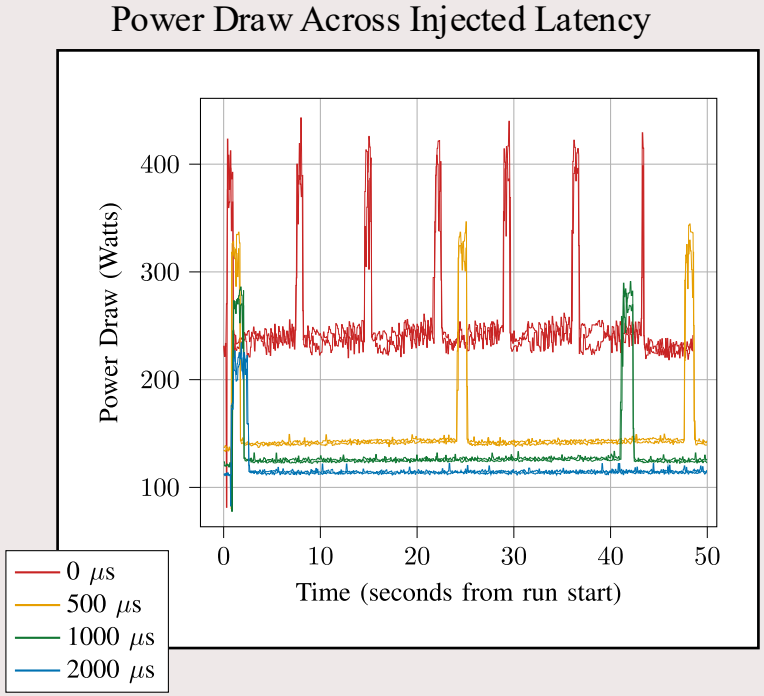
$$\text{Total sequence cache} = S \times \text{KV bytes per token}$$

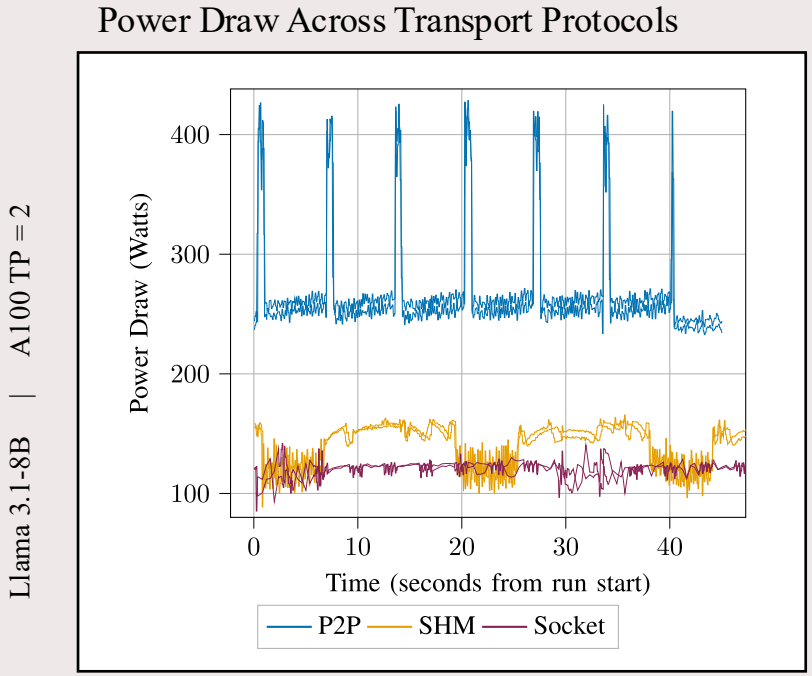
- Approximate KV cache capacity using static formulas
- Peak activation and framework overhead vary, but typically about 5-10% of available memory

Estimated Saturation Across Models

Model	TP	A100 Capacity	V100 Capacity
Llama 3.2-1B	1	1113	874
	2	2305	1828
	4	4689	3735
Llama 3.1-8B	1	139	79
	2	437	318
	4	1033	795
Llama 2-13B	1	6	0
	2	54	35
	4	149	111

Power Profiles

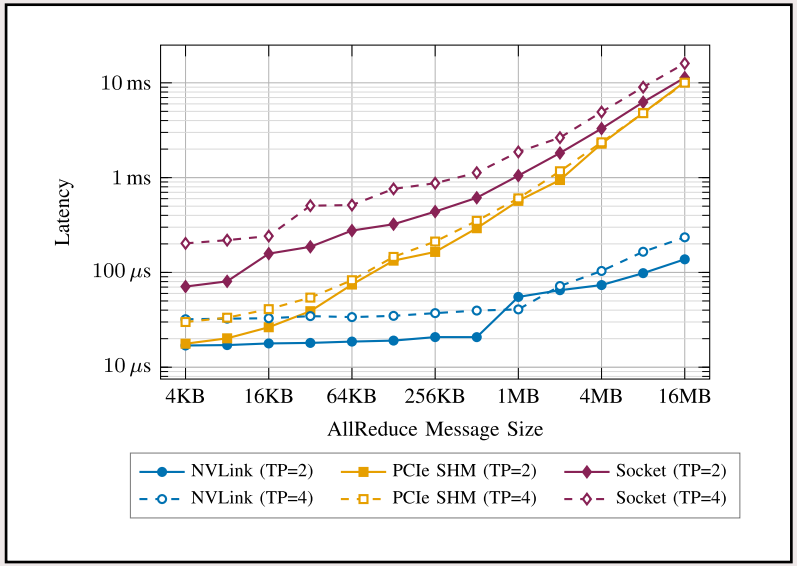




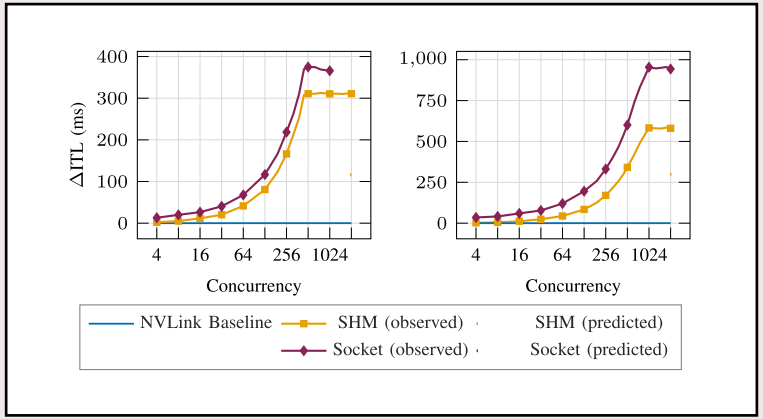
- GPUs waiting on communication idle
- Lower instantaneous power draw, but longer execution time leads to more expensive inference.

Microbenchmark Comparison

AllReduce Latency on A100s using `nccl-tests`



Predicted vs Observed ΔITL

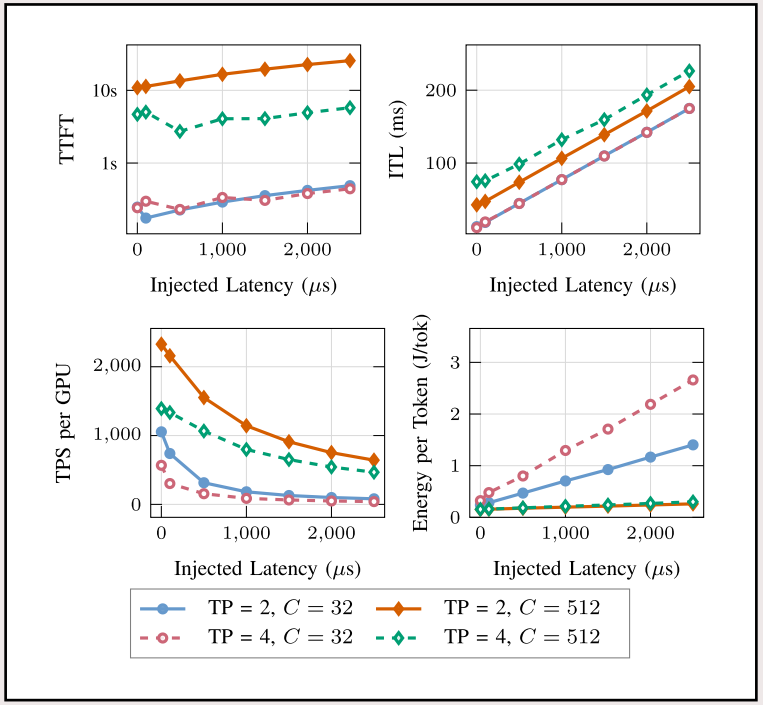


A100 System | Llama 3.1-8B

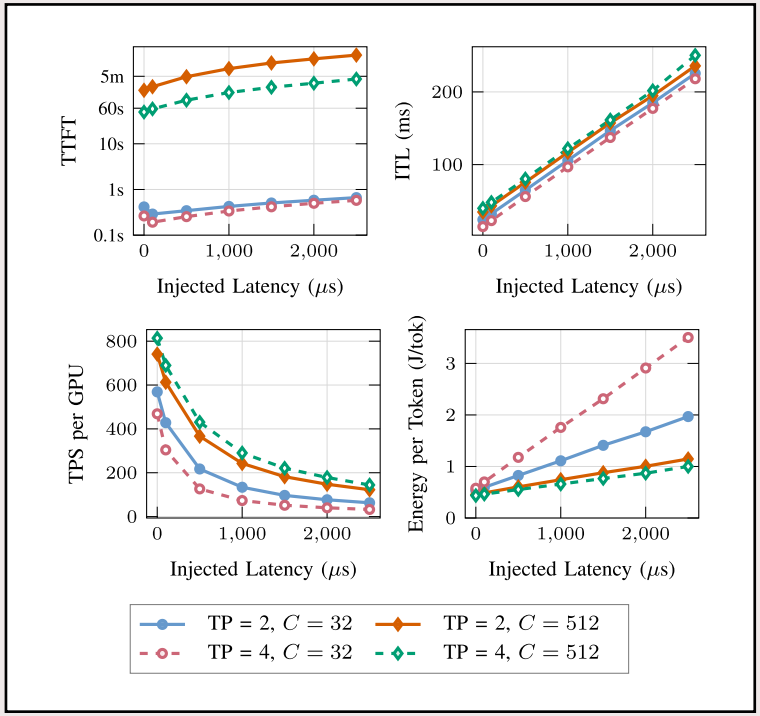
Predicted AllReduce latency decode step overhead: $\Delta\text{ITL} = 2L \times \Delta\text{AllReduce}$

- Combining standalone AllReduce microbenchmarks and the linear delay model, we can estimate and compare difference in ITL between NVLink and lower-tier transports
- This underpredicts actual communication overhead by up to 14−66%.

AllReduce Latency Injection: Llama 3.1-8B & 2-13B



Llama 3.1-8B



Llama 2-13B

References

[1] P. Foley, “NVIDIA DGX-1: The fastest deep learning system,” NVIDIA Technical Blog, 2016. [Online]. Available: https://developer.nvidia.com/blog/dgx-1-fastest-deep-learning-system/
[2] HAPPYWARE Server Processors GmbH, “GPU computing solutions: HPC servers & cluster infrastructure.” [Online]. Available: https://happyware.com/uk-en/it-solutions/hpc-high-performance-computing/gpu-computing
[3] B. Hanindhito, B. Patel, and L. K. John, “Bandwidth characterization of deepspeed on distributed large language model training,” in *2024 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, Indianapolis, IN, USA, May 2024.
[4] H. Qi, L. Dai, W. Chen, Z. Jia, and X. Lu, “Performance characterization of large language models on high-speed interconnects,” in *2023 IEEE Symposium on High-Performance Interconnects (HOTI)*, Virtual Conference, Aug. 2023.
[5] G. Mi, X. He, and X. Zheng, “Relevance of latency in ethernet networking for AI infrastructure,” in *ECOC 2024; 50th European Conference on Optical Communication*, Frankfurt, Germany, Sep. 2024.
[6] S. Verma and N. Vaidya, “Mastering LLM techniques: Inference optimization,” NVIDIA Technical Blog, 2023. [Online]. Available: https://developer.nvidia.com/blog/mastering-llm-techniques-inference-optimization/